



CITS5017 Deep Learning

Semester 2, 2026

Lab Sheet 2

(Not assessed)

1. Outline

This labsheet is a good exercise to practise what you learned from the lecture notes for Topics 1 and 2 (Chapter 10 “*Introduction to Artificial Neural Networks*” and Chapter 11 “*Training Deep Neural Networks*” of the textbook).

2. Tasks

Task 1

Continue your experiments on the *TensorFlow playground* (<https://playground.tensorflow.org>) from lab-sheet 1. This is a handy neural network simulator built by the TensorFlow team. There are four sets of data patterns for binary classification and two datasets for regression problems. Noise can be added to the data, batch size can be specified, and the numbers of hidden layers and of neurons can be varied, etc. For the *spiral* dataset, you will need around 4 hidden layers with 8 neurons each if you use only the first two features, x_1 and x_2 . Notice that training takes much longer for this dataset and often gets stuck on plateaus for long periods of time. This is the *vanishing gradients* problem, which can be alleviated with better weight initialization and other techniques, better optimizers, etc. Unfortunately, we cannot try all of these on the website.

The website allows you to gain some insights, without doing the coding, on how neural networks work and what the hyperparameters do.

Task 2

Go through the code supplied by Aurélien Géron for Chapter 11 (<https://github.com/ageron/handson-ml3>). In particular, focus on the *Reusing Pretrained Layers* section onward (how `model_B_on_A` was trained using `model_A`) and the validation accuracies when different optimizers were used for training 10 epochs.

Task 3

(Exercise 10 from Chapter 10 of the textbook) Train a deep MLP on the MNIST dataset (you can load it using `tf.keras.datasets.mnist.load_data()`). See if you can get over 98% accuracy by manually tuning the hyperparameters. Try searching for the optimal learning rate by using the approach presented in Chapter 10 (i.e., by growing the learning rate exponentially, plotting the loss, and finding the point where the loss shoots up). Next, try using early stopping, and plot learning curves using TensorBoard.